transforms v3: review and proposal

2026-11-10 · Baseline: master `7cc2a65` (v2.3.0 plus docs-only commits) · Nothing is implemented. Recommendations, not decisions, except item 10 (decided on review).

IN ONE PARAGRAPH

The existing v3 notes identify the right dangers but propose fixes that are bigger than the dangers. This review proposes a smaller 3.0: three behaviour fixes that only a major version can make, one clearer error diagnosis, one new insert error, three small error-shape repairs and four deletions. It adds one public type and one error variant, and removes more than it adds. It is worth doing after 2.4 and is not urgent, because no known user has hit any of the problems it fixes.

1 Background in one minute
2 How the review was done
3 The proposed 3.0 at a glance
4 The ten changes in detail
5 Considered and left out
6 What can ship in 2.x
7 Release plan
8 Decisions for the maintainer
9 Corrections to earlier documents
10 Limits of this review
11 Prototypes

1 Background in one minute

TERM	MEANING
Frame	A named point of view, such as `map`, `base` or `camera`. A coordinate means something only relative to a frame.
Transform	The recipe for converting coordinates from one frame to another at one instant: a shift (translation) plus a turn (rotation).
Quaternion	Four numbers that encode the turn. It is a pure turn only when its length (norm) is exactly 1. Otherwise it also stretches or shrinks.
Registry	The library's store of transforms over time, arranged as a tree of frames. Each link in the tree is a *hop*.
Lookup	Asking "how do I convert from frame A to frame B at time t?". The registry chains the hops between A and B and interpolates between stored samples.
Silent wrong answer	A result that looks valid and is not. The repository ranks this as worse than an error or a crash. Every item below is judged against it first.

The repository's scope rule: a capability belongs in the core only if callers cannot build it correctly on the public API, or if the version they would build is a source of wrong answers.

2 How the review was done

IN PLAIN TERMS

Ten reviewers each examined the design from one angle and ran experiments against the real code. A second, independent reviewer then tried to disprove each finding. A draft proposal was attacked by five critics, and its three most contested changes were built as throwaway prototypes and run against the existing tests.

TECHNICALLY

The request listed six lenses. Four were added before starting, because the six left them uncovered.

Requested (6): performance; safety; ergonomics; error handling and feedback; fixes and improvements over v2; current users and their integration.

ADDED LENS	WHY
Embedded targets, scalar type, determinism	The crate claims `no_std` parity on four bare-metal targets
Time, topology and lifecycle safety	Split from geometry: the dependent study puts the reachable silent answers there
Prior art	Web research on tf2 and peer crates
Migration, wire format, release strategy	Includes coherence of the v3 documents with each other

- **Round 1:** one reviewer plus one skeptic per lens (20 agents). Statuses reported here are the skeptics'.
- **Round 2:** five critics (minimalism, alternative designs, dependent cost from real sources, demand search on GitHub, consistency against code and `AGENTS.md`) and three prototypes (8 agents).
- **Baseline measured on host x86-64:** a lookup takes 60–300 ns over 1–4 hops, an insert about 115 ns, and a stored sample occupies 122 bytes of live memory.

3 The proposed 3.0 at a glance

`LOUD` means the compiler reports it. `SILENT` means code compiles and behaves differently, so it must be listed in the migration guide. `WIRE` means serialized data is affected.

#	CHANGE	KIND	WHO NOTICES
1	`get_transform_at` returns a cross-time type	`LOUD`	roslibrust_transforms: one public signature, three test lines
2	Rotations normalized at construction	`SILENT` `WIRE`	Near-unit rotations read back normalized; results change by up to parts per million
3	`inverse` is the conjugate	`SILENT` `LOUD`	Last-bit changes; one error variant removed
4	Lookups never return a non-finite value	`SILENT`	`Ok(inf)` becomes `Err(NonFiniteValues)`
5	`Disconnected` reported before `NotFoundAt`	`SILENT`	Callers that retry on `NotFoundAt`; no studied user matches a lookup variant
6	Error-shape repairs	`LOUD`	Patterns need `..`; the optional rename touches one match arm in transforms_io
7	Delete `Point`, `Transformable`, `Localized`, `get_transform_for`	`LOUD` `WIRE`	No published dependent found; `Point` payloads stop decoding
8	Delete `Transform::interpolate`	`LOUD`	No user found
9	Delete `Timestamp::as_seconds`	`LOUD`	No user found
10	An insert the history window discards returns `SampleExpired`	`SILENT`	Every adapter that sets `max_age`; a dimos batch fails as a whole

Left alone: `Stamp` and its serde tags, the flat `RegistryError`, the strict tree and `reparent_frame`, `remove_frame`, no extrapolation, `libm` everywhere, `BTreeMap` history, `latest_common_time`, `Registry::new` and `with_max_age`, the operators, the Non-Goals, MSRV 1.85 and edition 2024. No type name, module path

or constructor name changes, because they are re-exported down a chain (transforms_io, then pelorus_syncio).

4 The ten changes in detail

1 A separate type for cross-time answers

> **IN PLAIN TERMS**
>
> "Where is the object the camera saw half a second ago, relative to where the gripper is now?" That answer spans two instants. Today the library returns it in the same container as an ordinary single-instant transform, labelled with only one of the two times. Nothing stops a program from storing it back as if it were a measurement. The proposal gives it its own container that can be read, and cannot be stored or combined.

TECHNICALLY

```
// 2.3: a Transform stamped with target_time only
let tf = registry.get_transform_at("target", t2, "source", t1, "world")?;
receiver.add_transform(tf)?;      // Ok: a two-instant answer is now a sample at t2

// 3.0: geometry::CrossTimeTransform<T>, same method name and argument order
let x = registry.get_transform_at("target", t2, "source", t1, "world")?;
x.target_frame();  x.target_time();
x.source_frame();  x.source_time();
x.translation();   x.rotation();  // Vector3, Quaternion
receiver.add_transform(x);        // does not compile
```

- **Shape.** Private fields; derives `Debug`, `Clone`, `PartialEq`. No `Mul`, `inverse`, serde, conversion into `Transform`, `fixed_frame` field or apply method.
- **What it closes.** Accidental insertion and composition. It does not stop deliberate forging through `Transform::new`, which is no worse than forging any sample.
- **Why accessors.** Every studied user converts results into its own math types.
- **Why not delete the method.** A rebuild on the 2.3 public API was bit-identical on 7 of 9 cases, failed with `SameFrameMultiplication` when an endpoint equals the fixed frame, and needed one leg restamped through `Transform::new`, which defeats the equal-stamp guard. That is a wrong-answer generator, so the scope rule keeps it in the core.
- **Open detail.** A leg error can put the caller's target in a `source_frame` field (`src/core/registry/mod.rs:1101-1113`). Decide the payload when the type is built.

2 Rotations are made unit where they are built

> **IN PLAIN TERMS**
>
> A rotation is stored as four numbers whose combined length should be exactly 1. If the length is slightly more than 1, the rotation also enlarges whatever it turns. The library accepts rotations up to one millionth off and stores them as given, so each link in a chain stretches the result a little, and reading the chain backwards does not undo the stretch. The proposal rescales every rotation to length 1 at the moment a transform is created.

TECHNICALLY

```
// Accepted today (tolerance 1e-6) and stored as given:
let q  = Quaternion::from_wxyz(1.000_000_9, 0.0, 0.0, 0.0);
let tf = Transform::new("a", "b", Vector3::zero(), q, Stamp::At(t))?;
// rotate_vector computes q·v·q*, which scales v by |q|² = 1 + 1.8e-6
```

MEASURED EFFECT OF EDGE-OF-BAND ROTATIONS	SCALE ERROR
One hop	+1.8e-6
Four hops	+7.2e-6
Random forest of such edges, relative error on translations	up to 2.6e-5 (2.6 mm at 100 m)
For comparison: floating-point drift per composition	4e-17

The hazard is the accepted band, not drift. Quaternions from `f32` sources sit about 1e-8 to 1e-7 off unit norm (arithmetic, not measured on data), so most ROS traffic is stored unnormalized today. Forward lookups scale and inverse lookups normalize, so a round trip does not return its input.

```
// Prototype: called by Transform::new, static_between and Deserialize, after validate()
const UNIT_BAND: f64 = 8.0 * f64::EPSILON;

fn unitized(mut self) -> Self {
    let q = self.rotation;
    let norm_sq = q.w * q.w + q.x * q.x + q.y * q.y + q.z * q.z;
    if (norm_sq - 1.0).abs() > UNIT_BAND {
        self.rotation = q.normalize().unwrap_or(q);
    }
    self
}
```

- **Why the dead band.** Normalizing is not idempotent in floating point: a second pass changes bits in 15.6% of already-unit inputs (1e7 samples). Leaving values alone inside `8 × EPSILON` makes rebuilding a built value a fixed point. The observed maximum after one normalization was 4 EPS over 3e7 samples; the first-order bound of about 7 EPS is unproven.
- **Alternatives probed and rejected.** Normalizing only inside composition leaves single hops at +1.8e-6. A scale-invariant `rotate_vector` leaves composed rotations at +3.6e-6, past the insert tolerance. A public `Rotation` newtype closes the same paths and breaks every construction site. A tighter tolerance rejects `f32`-sourced data.
- **Declared silent change.** For in-band inputs, `tf.rotation()` no longer equals the quaternion passed in, and round-trip bytes change. For exactly normalized inputs, 28% of lookups in the forest probe changed last bits (from item 3). A single-hop lookup at a stored stamp still returns the stored transform bit for bit.
- **Not included.** A Newton correction per composition. Lookup chains drift about 1.8e-15 at depth 20, and a user loop over `*` needs about 2e10 compositions to reach the insert tolerance.

3 `inverse` becomes the conjugate

> IN PLAIN TERMS
>
> Reading a transform backwards requires undoing its rotation. For a rotation of length exactly 1, undoing it means flipping three signs. Today the library first rescales the rotation as a precaution, which costs a square root and can fail. Once item 2 guarantees length 1, the precaution has nothing left to guard.

TECHNICALLY

```
// src/geometry/transform/isometry.rs
- let rotation = self.rotation.normalize()?.conjugate();
+ let rotation = self.rotation.conjugate();
```

- Removes `TransformError::QuaternionError`. `Transform::inverse` stays fallible with `NonFiniteValues`.
- An inverting one-hop lookup was 10–13% faster in the prototype (host).
- Retires the `AGENTS.md` sentence "anything that inverts a rotation must normalize first".
- **Residual.** `Buffer::insert` validates and does not normalize. A derived transform that a caller drifted to within 1e-6 and re-published is inverted with up to 2e-6 relative error.

4 No lookup returns a non-finite value

> **IN PLAIN TERMS**
>
> With extreme numbers, arithmetic overflows to "infinity". Today some lookups return an answer containing infinity and mark it as success. A controller that receives "infinity metres, OK" is worse off than one that receives an error. The proposal checks the answer once, just before it is returned.

TECHNICALLY

```
// 2.3: overflow while composing a chain
registry.get_transform("left", "right", t)   // Ok(Transform { translation: inf, .. })
// 3.0
registry.get_transform("left", "right", t)   // Err(RegistryError::NonFiniteValues)

// Prototype: one check at the exit of get_transform and get_transform_at
let isometry = Self::resolve(target, source, timestamp, &self.data)?.ensure_finite()?;
```

- The documentation names only ancestor-ward lookups. Probes show sibling lookups return `Ok(inf)` in both directions, and `get_transform_at` has no check at all.
- Non-finite values are sticky through the arithmetic, so one exit check is exact. Its cost was within timing noise.
- **Not on `*`.** A check there closes no lookup hazard, and the tests deliberately create infinite transforms through `*` to exercise the insert check.
- Replaces the notes' translation bound `B = 1e100`, which needs a constant, narrows decoding and adds a second rule for `Point`.
- This is the one item a maintainer ruling could move to 2.x as a bug fix.

5 `Disconnected` is reported before `NotFoundAt`

> **IN PLAIN TERMS**
>
> A lookup can fail because the two frames are not connected at all, or because they are connected and no data covers the requested moment. The second reason invites waiting for more data. The first does not. Today, for two unconnected frames, the library can report the second reason, so a program that waits will wait forever.

TECHNICALLY

```
// Two trees with no link:   map → base      rig → camera
registry.get_transform("base", "camera", t)
// 2.3, when a hop misses t:
//   Err(NotFoundAt { frame, requested: t, covered: Some((start, end)), .. })
// 3.0:
//   Err(Disconnected { target_frame: "base", source_frame: "camera" })
```

- Precedence becomes `UnknownFrame > Disconnected > NotFoundAt`. `latest_common_time` already decides this way.
- **Minimal change:** delete the `.or_else(|| walk_failure.take())` fallback in `diagnose_not_found` (`src/core/registry/mod.rs:963`).
- **Declared silent change.** A caller that retries on `NotFoundAt` stops on a disconnected pair. The flip also applies to the legs of `get_transform_at`.

The resolver rewrite is separate and optional

A clean topology-first resolver deletes about 240 library lines and is bit-identical on values over more than 1M comparisons. Its only behavioural difference is the error flip above. Its performance on host is mixed:

CASE	AGAINST 2.3
Interpolated lookup that skips a long shared trunk	up to 3.7× faster
Interpolated lookup into an inner ancestor	1.2–1.8× faster
Exact-stamp root-ward chain	14–17% faster
Interpolated root-ward chain, 4–64 hops	9–17% slower
Exact-stamp lookup into an inner ancestor near a deep leaf	1.4× slower
Allocations per lookup	count unchanged; bytes down 55–73%

Take it only as a deletion, and only if a variant recovers the root-ward cost. A 2.x fast path with a fallback is rejected: two resolvers can diverge.

6 Error shapes

> **IN PLAIN TERMS**
> Errors are structured records. Three small repairs: let those records gain fields later without breaking users' code, stop one error from printing two timestamps that look identical when they differ, and optionally give one error a name that matches what it means today.

TECHNICALLY

```
// (a) Struct variants become #[non_exhaustive]: a field can be added in a minor
#[non_exhaustive]
Disconnected { target_frame: String, source_frame: String },
// downstream patterns then need `..`:
RegistryError::Disconnected { target_frame, source_frame, .. } => { /* … */ }

// (b) TimestampMismatch loses its payload
//     the f64 seconds print as equal for stamps 1 ns apart at epoch scale
- TimestampMismatch { lhs: f64, rhs: f64 },
+ TimestampMismatch,

// (c) Optional rename
- ReparentingNotSupported { .. }
+ ParentMismatch { .. }
```

- **(a)** applies to `Disconnected`, `NotFoundAt`, `NoCommonTime` and the parent-pin variant in `RegistryError`, and to `IncompatibleFrames` and `SameFrameMultiplication` in `TransformError`. Tuple and unit variants stay frozen.
- **(b)** After items 7 and 8 the only producer is `*`, whose caller holds both operands. This replaces the notes' `TransformError<T>`, which adds a type parameter for one variant and breaks roslibrust's public conversion trait.

- **(c)** "Not supported" misleads now that `reparent_frame` exists, and the variant's rustdoc defers the rename to 3.0. It is the one known break in transforms_io (`mt_pubsub_backend.rs:342`). Cosmetic; it carries no weight in the case for 3.0.

7 Delete `Point`, `Transformable`, `Localized`, `get_transform_for`

> **IN PLAIN TERMS**
>
> The library ships a helper that bundles a position with a frame label and can have a transform applied to it. No user of it was found. It does not check its own inputs, so an invalid position passes through as success. Its name is misleading, because it also carries an orientation. Applying a transform by hand is one line, so the helper is removed.

TECHNICALLY

```
// 2.3
let tf = registry.get_transform_for(&point, "base")?;
point.transform(&tf)?;

// 3.0: rotate first, then translate
let tf = registry.get_transform("base", "sensor", t)?;
let p_base = tf.rotation().rotate_vector(p_sensor) + tf.translation();
```

- **Evidence.** No published dependent calls them (grep of six users' sources; GitHub code search found none, which is weak evidence). A NaN position or a zero quaternion goes through `Point::transform` as `Ok`. It allocates a `String` per element. Each trait has one implementor.
- **Demand that exists.** Three people used `Point` in 0.3.x and 1.x issues (#43, #45, #65/#66), and #76 wanted a helper. The same threads show `Point` confusing its users.
- **Coupling.** With no library-side apply, users rotate by hand. Item 2 is what makes that safe.
- **Obligations.** Rewrite the README headline example, re-point `tests/golden_vectors.rs` and `tests/properties.rs:205` without touching the expected numbers. `Point`'s pinned wire format stops decoding.
- **Dissent.** The geometry lens would harden `Point` instead (private fields, fallible constructor). That adds surface for a type no dependent uses.

8 Delete `Transform::interpolate`

> **IN PLAIN TERMS**
>
> Interpolation estimates a pose between two measurements. The registry does this internally. There is also a public function for doing it by hand, which the registry never calls and no user was found calling. It is the only source of three error cases, which disappear with it.

TECHNICALLY

```
Transform::interpolate(&from, &to, timestamp)   // removed
```

- The buffer interpolates on the crate-private `Isometry`. This function is the only producer of `StaticInterpolation`, `TimestampOutOfRange` and the `"a -> b"` string form of `IncompatibleFrames`.
- It can be rebuilt on the public `Quaternion::slerp` plus a linear blend (not probed).
- `tests/properties.rs:751` uses it as the oracle for registry interpolation and needs an in-test reference instead.

9 Delete `Timestamp::as_seconds`

> **IN PLAIN TERMS**
>
> This function converts a nanosecond count to seconds and refuses whenever precision would be lost. Precision is lost for any time later than 104 days after 1970, so it fails on every real clock reading. A sibling function that converts anyway, and says so in its name, stays.

TECHNICALLY

```
Timestamp::now().as_seconds()         // Err(AccuracyLoss) above 2^53 ns ≈ 104 days: removed
Timestamp::now().as_seconds_lossy()   // f64: stays
```

- `TimeError::AccuracyLoss` stays. A custom clock uses it, and `tests/time_traits.rs:277,317-329` pin that path.

10 An insert the history window discards is an error

> **IN PLAIN TERMS**
>
> A registry can be told to keep only the last N seconds of history. If one measurement arrives stamped far in the future, from a glitch or a clock jump, the registry treats that as "now" and discards everything older. From then on every normal measurement counts as too old and is thrown away, while the library reports success each time. The proposal makes the library say so: a measurement that would be discarded on arrival is rejected with an error that names the frame. A batch of measurements is accepted or rejected as a whole.

TECHNICALLY

```
// at(s): a sample of map → base stamped s seconds
let mut r = Registry::with_max_age(Duration::from_secs(10));
r.add_transform(at(100))?;          // stored
r.add_transform(at(1_000_000))?;    // glitch: accepted, evicts everything older
r.add_transform(at(101))            // 2.3: Ok(()), stored and evicted at once
                                    // 3.0: Err(SampleExpired { frame: "base", stamp, newest })

// Batches equal the insert loop, so order inside a batch matters (fresh registry each):
r.add_transforms([at(50), at(100)])   // Ok: 50 is stored, then evicted by 100
r.add_transforms([at(100), at(50)])   // Err(SampleExpired): nothing is stored
```

Added on maintainer review, 2026-11-10. Three lenses had ranked this the most serious silent defect outside the ledger; the stress test turned the fix down. Its reasons are kept below as accepted costs.

- **Boundary.** The eviction boundary: a sample exactly at newest minus `max_age` is accepted, a strictly older one is rejected, before any mutation. A registry without `max_age` never reports it. Static frames are unaffected, and the `reparent_frame` seed is exempt because it replaces the history.
- **Batches fail as a whole** (maintainer decision). `add_transforms` stays atomic. Each member is checked against the frame's running newest stamp, which includes earlier members of the same batch.
- **Alternative considered: check against the batch's newest stamp.** That rule rejects `[at(50), at(100)]` too. It is independent of order, and it refuses a batch that carries a future-stamped glitch after a normal sample, which the loop rule accepts. It was not taken because the batch would stop equalling the loop: the `add_transforms` rustdoc promises the state and the first error of inserting in order, and a property test (`tests/properties.rs:640`) pins it. Both stay unchanged under the loop rule.
- **Why an error.** The crate never consults a wall clock, so it cannot tell a future-stamped glitch from new data and cannot refuse the poison itself. Reporting is the only thing the core can do, and `Ok(())` for a sample that is not stored is a false return value. `newest` lets a caller that has a clock tell a late sample

from a poisoned edge. The check is buildable outside for one insert, but a batch needs the running-newest logic, which is the copy that goes wrong.

- **What it does not do.** The poison insert is still accepted; the error fires on the next normal insert. A jump smaller than `max_age` still interleaves two epochs silently, so the docs must not call this reset detection. Lookups do not change: one driven by `latest_common_time` keeps serving the frozen pose, and one at a real stamp already fails with `NotFoundAt`.

Costs accepted

- Routine late arrival beyond `max_age` becomes `Err`, which aborts an insert loop that uses `?`.
- One expired member rejects a whole TF message in dimos (PR #4722, open), where 2.3 stores the rest. After a poisoned frame, healthy frames in the same message stop updating until the caller filters by the `frame` in the error and retries.
- transforms_io and roslibrust_transforms log per rejected transform and expose no reset call, so they log once per message.
- The error does not carry the rejected transform. A caller that wants to retry clones before the call.
- `Buffer::check` reads stored state. Its rustdoc invariant (`src/core/buffer/mod.rs:343`) no longer holds as written.
- This reverses the dependents addendum, which voted 0 of 3 against an `Expired` error.

Recovery and compatibility

- **Recovery needs no new API.** `remove_transforms_before` past the newest sample drains the buffer and resets its expiry reference; it is registry-wide. Two `reparent_frame` calls clear one frame.
- **Compatibility.** `RegistryError` is `#[non_exhaustive]`, so nothing stops compiling. It is 3.0 only: the `with_max_age` rustdoc describes eviction on insert, and three tests pin the old contract.
- **Prototype.** +106/−9 library lines including `clear_history`; the batch equals the insert loop, including the first error and unchanged state on error (proptest, 2000 cases).

5 Considered and left out

Each row is an item from the v3 notes or ledger, or an idea raised during this review.

IDEA	IN PLAIN TERMS	WHY IT IS LEFT OUT
Public `Rotation` and `Isometry` types	A dedicated "guaranteed length-1 rotation" type, enforced by the compiler.	Items 2–4 close the same hazards with no new type. About 17 construction sites in dependents would break (count not re-verified).
Translation bound `B`	Refuse any distance above a huge fixed limit, so sums cannot overflow.	Item 4 is exact and needs no constant. `B` narrows decoding.
Newton renormalization	Nudge the rotation back to length 1 after every multiplication.	Not needed for lookups. Changes bits on 28% of identity compositions and diverges for norms above √3.
`Resolved` result type	A new container for every lookup answer that says "target" and "source".	A second type that closes nothing. Only the cross-time type needs those words.
Operators to named methods	Replace `a * b` with `a.checked_mul(b)`, since the operators return `Result`.	Closes no hazard. About 60 sites in this repository and four roslibrust doctests. *Dissent:* the ergonomics lens calls the operators the least idiomatic part of the API.
Frame names on insert errors	Make every "insert rejected" error say which frame it was.	A single-insert caller holds the transform. Only `add_transforms` is blind, and its one user is an unmerged PR.

`TransformError<T>`	Carry exact timestamps in geometry errors.	Replaced by the unit `TimestampMismatch` (item 6).
`TransformError` arm to `Time(TimeError)`	Narrow one catch-all error arm.	Total only once registry composition is infallible, which needs the resolver rewrite. `tests/time_traits.rs:317` pins the current arm.
Required `max_age`	Force every user to say how much history to keep.	Bounds memory only: an 8 s dropout is still interpolated inside a 10 s window. Every studied user already passes `with_max_age`. Document 122 B per sample instead.
`clear_history`	One call that empties the stored history after a clock reset.	Additive, so not a v3 item. Recovery is buildable today (item 10). A registry-wide empty clear is not buildable for a generic time type when a sample sits at the type's maximum. Revisit on a request.
Maximum interpolation gap	Refuse to estimate between two measurements that are too far apart.	Additive, so never a v3 item. No design and no request. A time bound also misses the exact hazard, which is more than 180° between adjacent samples.
`f32` or a scalar parameter	Let small chips use cheaper, less precise numbers.	A Non-Goal. Staying `f64` with no scalar parameter keeps a concrete `f32` module additive later.
`FrameId`, interning	Replace frame-name strings with numeric handles for speed.	Hashing is 8–16% of a hop. `Arc` does not build on thumbv6m or riscv32imc.
`get_latest`, `can_transform`, bulk lookups, caches, typed frames	Convenience shortcuts familiar from tf2.	Buildable on the public API, or a new source of wrong answers.
Self-edge and empty-name checks in `Transform::new`	Reject nonsensical frame names earlier.	`get_transform(x, x)` returns an `(x, x)` identity that must stay constructible. Both checks stay at insert.
Euler and axis-angle constructors; `mint`, `nalgebra`, `glam` features	More ways to write a rotation; bridges to other math libraries.	Buildable outside. No dependent builds quaternions from angles.
Removing `approx` or `thiserror`	Fewer dependencies.	Unrelated to v3. approx 0.6 has been at rc2 since 2026-02-05; decide when it is stable.

6 What can ship in 2.x

> **IN PLAIN TERMS**
>
> These fixes break nobody's code, so they need not wait for a major version. Most are documentation.

- Reject empty frame names, and the documentation corrections. Both are already in the 2.4 proposal.
- Document the silent drop of expired inserts, with the recovery recipe from item 10. 2.x keeps the drop; item 10 replaces it in 3.0.
- One sentence for the embedded claim: `f64` runs in software on every gate target unless a Cortex-M7 with a double-precision FPU is built with `target-cpu=cortex-m7` (assembly probe: 36 `__aeabi_dmul` calls by default, none with the flag).
- The `latest_common_time` example unwraps `.at()`, which panics when copied onto an all-static chain. Show the `match`.

- A rustdoc recipe for "can waiting fix this error", worded as likely and not certain: a late backfill can make an older instant servable.
- Internal cleanup: drop `Buffer.child`, `InsertError::ChildFrameMismatch` and its unreachable `From` arm; store an `Isometry` in the static slot.
- `#[deprecated]` on `Point`, `Transformable`, `Localized` and `get_transform_for`, linking to a discussion, as the one cheap way to hear from users nobody found. **Needs sign-off:** the gate runs clippy with `-D warnings`, so in-repo uses need `#[allow(deprecated)]`.

7 Release plan

> **IN PLAIN TERMS**
> Ship 2.4 first. Build 3.0 on a branch and pass the full test gate. Publish one release candidate and compile a real dependent against it. Then release. Old serialized transforms keep loading. Five of the ten changes alter behaviour without a compiler error, so the migration guide must list them by name.

TECHNICALLY

- **Order.** 2.4, then the 3.0 prototype through `tests/test_all.sh`, then one `3.0.0-rc` built against transforms_io, then the release.
- **Case for yes.** Item 2 changes values for most real traffic. That only gets more expensive to change as users arrive. The warrant report's two triggers do not hold up: the hardware trigger contradicts v3-3 §6, and the deprecation trigger cannot fire under the 2026-11-03 stance.
- **Wire.** Postcard layouts and JSON names stay frozen, and 3.0 reads every 2.x `Transform` payload. Near-unit rotations are normalized on decode, and `Point` payloads stop decoding. Bit-exact JSON round trips need `serde_json`'s `float_roundtrip` feature regardless of this change (its default parser is off by one ulp on 10.5% of random values).
- `tests/v2_compatibility.rs`**.** Three of nine tests flip: disconnected precedence (item 5), the rotation half of the exact-sample test (item 2), and the insertable cross-time result (item 1).
- **Other pinned tests that flip.** `an_overflowing_lookup_reports_the_flat_variant_only_where_it_inverts`, `inverse_renormalizes_a_rotation_that_drifted_off_the_unit_sphere` and `add_transform_rejects_a_republished_chain_that_left_validity`. Item 10 flips `out_of_order_insert_does_not_regress_latest_timestamp`, `zero_max_age_out_of_order_insert_is_ok_but_immediately_evicted` and `dynamic_history_matches_a_reference_map_after_upserts_expiry_and_cleanup`. Golden vectors, slerp bit pins, frozen postcard bytes and all 36 doctests passed unchanged on the item 2–4 prototype.
- **Acceptance.** Add a dual-version differential harness (2.3 beside 3.0 with an allow-list of intended flips) and a corpus of 2.3-encoded blobs decoded by 3.0. `cargo semver-checks` cannot see the silent rows.
- `MIGRATION.md`**.** One row per item, tagged loud, silent or wire. Silent: items 2, 3, 4, 5 and 10.
- `AGENTS.md`**.** Six invariant bullets need rewording, and the `Point` mentions in Architecture and Style go.

A policy sentence to decide

An earlier ruling let a minor release reject empty names, while the warrant report warns against silent behaviour changes in minors. One rule would classify every row:

> A minor may reject an input class the docs never promised to accept, when accepting it produced a wrong answer. A minor never changes the value returned for an accepted input, or a declared error precedence.

This relaxes the API-stability section of `AGENTS.md`, so it is the maintainer's to adopt.

8 Decisions for the maintainer

#	QUESTION	RECOMMENDATION
1	Cut a 3.0 of this size after 2.4?	Yes, not urgently
2	Delete `Point` and the two traits, or harden `Point`?	Delete; deprecate in 2.x first **(sign-off: `#[allow(deprecated)]` in the repo)**
3	Normalize at construction with an `8 × EPSILON` band?	Yes, with a proof or a pinned property test for the band
4	Lookup finiteness check: 3.0, or a 2.x bug fix?	3.0, unless ruled a bug fix like the empty-name case **(sign-off)**
5	Resolver rewrite after the precedence flip?	Only as a deletion that does not slow root-ward lookups
6	Rename `ReparentingNotSupported`?	Optional; take it only if 3.0 happens
7	Keep the operators?	Yes
8	Add an error for expired inserts?	**Decided 2026-11-10: yes**, in 3.0 (item 10); a batch fails as a whole. The review had recommended no.
9	Adopt the policy sentence on minors?	Yes **(sign-off: changes `AGENTS.md`)**

9 Corrections to earlier documents

> **IN PLAIN TERMS**
>
> Statements in earlier analysis documents that this review found wrong or imprecise. None is applied; each was confirmed by an experiment or by reading the code.

DOCUMENT	CORRECTION
v3-3, finding 2 and §1	`Ok(inf)` is not ancestor-ward only; sibling lookups and `get_transform_at` return it too. "Stored normalized" glosses over non-idempotent normalization.
v3-3, finding 10	The missing `alloc::sync` also affects riscv32imc, a CI target.
v3-3, §2	"target/source for anything a query returns" implies a new return type for `get_transform`, which conflicts with its own §7.
Warrant report, `Rotation` row	"Small user payoff" rests on drift. The accepted tolerance band is the effect.
Warrant report, triggers	See section 7: neither trigger can fire as written.
Warrant report, ergonomics item 3	The example comments "transform from camera to base" read consistently as mapping camera coordinates into base. That is a vocabulary difference, not drift.
Dependents README	"Per-edge clock-reset detection needs access the API does not expose" is wrong: a single-hop `latest_common_time` returns the edge's newest stamp.
Dependents study, dimos	PR #4722 was rebased on 2026-11-09; its head is `fc941c1`, not `92936a7`. It is still open.
Cross-time addendum	"An additive type helps only callers who migrate" is overstated: `#[deprecated]` warns every direct caller. Only transitive users are unprotected.
v3-2 and the perf notes	Live memory is 122 B per sample, not the 64 B payload: sequentially appended B-tree nodes stay about half full.

10 Limits of this review

> **IN PLAIN TERMS**
> What was not checked, so that no claim above is read as stronger than its evidence.

- **No hardware.** Nothing ran on a microcontroller or under QEMU. All timings are host x86-64. thumbv6m and thumbv8m are not installed locally.
- **No gate run.** `tests/test_all.sh` ran on no prototype. They ran `cargo test` with default and no default features; the item 2–4 prototype also ran with `serde`.
- **No dependent compiled.** Compile-error counts come from grep against this proposal's text.
- **Dead band unproven.** The `8 × EPSILON` width is empirical, and `libm::sqrt` was not checked on soft-float targets.
- **Prior art unverified.** Claims about tf2, sguaba, cu-transform and schiebung come from one web pass. No decision rests on them.
- **Private users are invisible.** crates.io lists three reverse dependencies.
- **Not covered by any lens:** lock contention under `RwLock`, allocator failure on small-RAM parts, and `SystemTime` as the time type in benchmarks.

11 Prototypes

Library-only diffs against `7cc2a65`, in `analysis/v3-review/prototypes/`. They are evidence, not patches: none is gate-clean, and none updates docs or tests.

FILE	CONTENT	LINES	EXISTING TESTS THAT FAIL
`a2-normalize-conjugate-finite-gate.diff`	Items 2–4, plus a check on `*` that is not proposed	+51 / −28	5, all intended; one from the `*` check
`a4-topology-first-resolver.diff`	Resolver rewrite of item 5	+75 / −300	1, the precedence pin
`sample-expired-clear-history.diff`	Item 10, plus `clear_history`, which is not proposed	+106 / −9	3, pins of the old contract

Full review text with file and line references: `analysis/v3-review/REPORT.md`.